

PROJECT PHASE-II SYNOPSIS

AI-Powered Automated Insurance Claim Estimator

Department: Artificial Intelligence and Machine Learning

Academic Year: 2026–27

Semester: VII

Project Phase: II

Guide: Dr. Dhananjaya V, HOD, Department of AIML

Project Team

Student	USN
Adithya S	1ST23AI002
Spandana G	1ST23AI042
Adarsh R	1ST24AI400
Harshith GU	1ST23AI014

Abstract

The **AI-Powered Automated Insurance Claim Estimator** is a web-based prototype that assists with preliminary vehicle-insurance claim assessment. The system collects vehicle and accident details, accepts vehicle-damage images, performs image-based damage analysis, classifies the observed damage as **Minor, Moderate, or Severe**, identifies a likely damaged vehicle part, recommends a suitable repair action, and estimates the repair cost using parts and labour pricing.

The application contains a React and Vite frontend and a Python FastAPI backend. The current prototype uses an explainable computer-vision baseline based on image edge and texture signals. The analysis response contains a potential damaged region, confidence score, damage type, severity, repair recommendation, and itemized cost. Multiple images can be processed, and repeated views of the same damaged part are consolidated so that the final estimate represents a repair job rather than a charge for every photograph.

The system aims to reduce manual effort, provide faster preliminary assessment, improve consistency, and give users a transparent claim summary. A trained YOLO-based detector is

identified as a future enhancement requiring a labeled vehicle-damage dataset and model evaluation.

1. Problem Statement

Vehicle-insurance claim assessment commonly depends on manual inspection by surveyors. This process can be time-consuming, expensive, difficult to scale, and subject to inconsistent judgments. Different assessors may estimate damage severity and repair cost differently, and claimants may have limited visibility into how an estimate was produced.

There is a need for a software system that can support the early stages of claim assessment by collecting structured claim information, analyzing vehicle images, classifying visible damage severity, recommending an appropriate repair action, and producing a transparent cost estimate.

2. Objectives

1. To develop a web-based vehicle-insurance claim creation workflow.
 2. To collect vehicle and accident information in a structured manner.
 3. To accept and validate vehicle-damage images.
 4. To analyze uploaded images using an explainable computer-vision baseline.
 5. To identify a likely damaged vehicle part and damage region.
 6. To classify damage severity into Minor, Moderate, and Severe categories.
 7. To classify visible damage into Scratch, Dent, Crack, Broken Component, or Deformation categories.
 8. To recommend a suitable repair action based on damage type and severity.
 9. To estimate parts cost, labour cost, and total repair cost.
 10. To consolidate multiple image results and avoid duplicate charges for the same damaged part.
 11. To generate a downloadable claim-assessment report.
 12. To provide a foundation for future YOLO-based deep-learning integration.
-

3. Scope of the Project

3.1 Included in the current prototype

- Claim creation interface
- Vehicle details form
- Accident details form
- Image upload with JPG, PNG, and WEBP validation
- One-to-eight image upload support
- Image size validation up to 10 MB per image
- Baseline image analysis using NumPy and Pillow
- Potential damaged-region detection
- Likely damaged-part estimation
- Damage-type classification
- Minor, Moderate, and Severe severity classification
- Repair recommendation generation
- Parts and labour cost estimation in INR
- Multi-image result aggregation
- Duplicate damaged-part handling
- Claim review page
- Downloadable JSON claim report
- FastAPI health and analysis endpoints

3.2 Outside the current prototype scope

- Production insurance claim approval
- Legal or financial decision-making
- User authentication and role management
- Persistent database storage
- Payment processing
- A production-trained YOLO model
- Certified repair-shop pricing

- Automatic legal claim submission
-

4. Software and Hardware Requirements

4.1 Software requirements

- Windows or Linux
- Python 3.10 or later
- Node.js and npm
- React
- Vite
- FastAPI
- Uvicorn
- Pillow
- NumPy
- python-multipart
- VS Code
- Git and GitHub

4.2 Hardware requirements

- Intel Core i5 processor or equivalent
 - Minimum 8 GB RAM
 - At least 256 GB storage
 - Internet connection for setup and repository access
 - GPU recommended only for future deep-learning model training
-

5. System Architecture

```
User
|
v
React + Vite Frontend
|
| Vehicle details, accident details, image upload
v
FastAPI Backend
|
v
Image Validation and Preprocessing
|
v
Baseline Computer-Vision Analysis
|
+→ Damage Region and Part Estimation
|
+→ Damage Type Classification
|
+→ Severity Classification
|
+→ Repair Recommendation
|
+→ Parts + Labour Cost Estimation
|
v
Claim Review and Downloadable Report
```

6. Methodology

Stage 1: Claim creation

The user starts a new claim from the home page. The system provides a guided three-step workflow.

Stage 2: Vehicle and accident details

The system collects manufacturer, model, manufacturing year, registration number, accident date, approximate time, accident location, accident type, involvement of another vehicle, vehicle drivability, and accident description.

Stage 3: Image collection

The user uploads one to eight images. The frontend validates the file type and size, generates previews, supports removal, and sends the images to the backend for analysis.

Stage 4: Image preprocessing

The backend reads the uploaded image, converts it to RGB, resizes it to a manageable maximum dimension, and converts it into a NumPy array. Grayscale signals are calculated for edge and texture analysis.

Stage 5: Damage-region estimation

The image is divided into a four-by-four grid. Local edge and texture signals are calculated for each region. The strongest region is selected as a potential damaged area and returned as a normalized bounding box.

Stage 6: Damaged-part estimation

The prototype maps the location of the strongest region to a likely vehicle part such as a front bumper, hood, fender, door panel, or body panel. This is an explainable baseline mapping and will be replaced by a trained detector in future work.

Stage 7: Damage-type classification

The system uses damage score, edge signal, and texture signal to classify the visible damage as one of the following:

- Scratch
- Dent
- Crack
- Broken component
- Deformation

Stage 8: Severity assessment

A normalized damage score is mapped to three categories:

Score range	Severity
Below 0.40	Minor
0.40 to below 0.68	Moderate
0.68 and above	Severe

Stage 9: Repair recommendation

The damage type is mapped to a recommended repair action:

Damage type	Recommendation
Scratch	Paint repair
Dent	Paintless Dent Repair (PDR)
Crack	Component repair or replacement
Broken component	Component replacement
Deformation	Professional inspection

Stage 10: Repair-cost estimation

The estimator uses a predefined part-price catalog and severity multipliers. The final result contains separate parts cost, labour cost, and total repair cost in INR.

Severity	Parts multiplier	Labour multiplier
Minor	35%	50%
Moderate	70%	85%
Severe	100%	130%

When several images show the same likely part, the system retains the highest severity for that part and does not charge the same part repeatedly.

Stage 11: Claim report

The review page displays the overall severity, per-image results, damaged part, damage type, recommendation, parts cost, labour cost, total estimate, and unique damaged parts. The user

can download a structured JSON claim-assessment report.

7. Functional Requirements

1. The system shall allow a user to create a new claim.
 2. The system shall collect vehicle and accident details.
 3. The system shall accept one to eight vehicle images.
 4. The system shall validate supported image formats and file sizes.
 5. The backend shall analyze submitted images.
 6. The system shall return a potential damage region and likely damaged part.
 7. The system shall classify damage type and severity.
 8. The system shall calculate parts, labour, and total repair cost.
 9. The system shall generate repair recommendations.
 10. The system shall display all results on a review page.
 11. The system shall allow the user to download a claim-assessment report.
-

8. Non-Functional Requirements

- **Usability:** The claim flow should be understandable to a non-technical user.
 - **Performance:** Normal image analysis should complete within a few seconds on a standard laptop.
 - **Reliability:** Invalid files should be rejected with a clear message.
 - **Maintainability:** Frontend and backend modules should remain separated.
 - **Transparency:** The result should show score, severity, recommendation, and cost components.
 - **Scalability:** The API design should allow a trained model and database to be added later.
 - **Security:** Production deployment should add authentication, secure storage, and privacy controls.
-

9. API Design

Health endpoint

```
GET /health
```

Returns the backend status and analysis-engine name.

Claim-analysis endpoint

```
POST /api/analyze-claim
```

Accepts multipart image files and returns:

- Overall severity
 - Overall and average damage scores
 - Per-image detections
 - Damage type
 - Likely damaged part
 - Repair recommendation
 - Parts cost
 - Labour cost
 - Total repair cost
 - Unique-part estimate summary
-

10. Testing and Results

The following tests were completed for the current prototype:

Test	Result
Backend syntax check	Passed
Backend health endpoint	Passed
One-image multipart upload	Passed
JPG/PNG/WEBP validation	Implemented
Damage-region response	Passed
Severity classification	Passed
Damage-type classification	Passed
Repair recommendation	Passed
Parts and labour calculation	Passed
Duplicate-part aggregation	Passed
Frontend production build	Passed
ESLint validation	Passed
Claim review navigation	Passed
JSON report download	Implemented

The current test implementation successfully completed the workflow from image upload through damage analysis, severity assessment, recommendation, and cost estimation.

11. Current Limitation

The current damage-analysis engine is an explainable baseline computer-vision implementation using edge and texture signals. It is not a trained YOLO model. Therefore, image lighting, reflections, shadows, background textures, and image composition can affect the estimated score.

The repair-price catalog is also a prototype catalog. Actual costs vary by vehicle model, location, workshop, part availability, labour rates, taxes, and insurer policy.

The system should therefore be described as a **prototype decision-support tool**, not as an automated insurance approval system.

12. Future Enhancement: YOLO Integration

A future version will integrate a trained YOLO-based detector. The planned steps are:

1. Collect a representative vehicle-damage dataset.
2. Annotate bounding boxes for damaged parts and damage regions.
3. Label damage types and severity categories.
4. Split the dataset into training, validation, and test sets.
5. Train and evaluate a YOLO model.
6. Measure precision, recall, mAP, and class-wise performance.
7. Integrate the trained weights into the FastAPI backend.
8. Replace the baseline part estimation with model predictions.
9. Compare YOLO results against the baseline.
10. Add model versioning and monitoring.

Other future enhancements include database storage, authentication, vehicle-model-specific pricing, PDF report generation, cloud deployment, and insurer/workshop integration.

13. Conclusion

The AI-Powered Automated Insurance Claim Estimator provides a complete working prototype for preliminary vehicle-damage claim assessment. It combines a guided claim workflow, image upload, baseline computer-vision analysis, damage-type classification, severity assessment, repair recommendation, itemized cost estimation, and downloadable reporting.

The project demonstrates how artificial intelligence and web technologies can support faster and more transparent insurance claim processing. The current implementation provides a foundation for integrating a trained YOLO model and a production-grade dataset in the next phase.